



UNIVERSITAT POLITÈCNICA
DE CATALUNYA
BARCELONATECH

Formal Guided Test Sequence Optimization

Lorenzo Pattaro Zonta

Processor Verification - 29 May 2026

Introduction

Verification process faces significant challenges:

Introduction

Verification process faces significant challenges:

- # of test sequences grows exponentially with # of test cases.
- Executing all possible sequences is often infeasible
- Identification of the most effective test sequences: crucial for efficient testing.

Agenda

1. Current Verification Approaches
2. Related Works
3. FGTSO Framework
4. Results

Current Verification Approaches

Current Verification Approaches

Formal verification:

Simulation-based verification:

Current Verification Approaches

Formal verification:

- Comprehensive design exploration.
- Encounters state space explosion problem for large scale designs.

Simulation-based verification:

Current Verification Approaches

Formal verification:

- Comprehensive design exploration.
- Encounters state space explosion problem for large scale designs.

Simulation-based verification:

- High scalability and applicability.
- Effectiveness rely on well-designed constraints.
- Biased constraints: miss critical scenarios.
- Strict constraints: increase debugging complexity.

Formal Guided Test Sequence Optimization

- Enhance coverage and verification efficiency.

Formal Guided Test Sequence Optimization

- Enhance coverage and verification efficiency.
- Leverage simulation scalability and formal verification precision to target critical conditions

Formal Guided Test Sequence Optimization

- Enhance coverage and verification efficiency.
- Leverage simulation scalability and formal verification precision to target critical conditions

Formal Guided Test Sequence Optimization

- Enhance coverage and verification efficiency.
- Leverage simulation scalability and formal verification precision to target critical conditions

Formal Guided Test Sequence Optimization

- Enhance coverage and verification efficiency.
- Leverage simulation scalability and formal verification precision to target critical conditions
- Integrate simulation and formal verification.

Formal Guided Test Sequence Optimization

- Enhance coverage and verification efficiency.
- Leverage simulation scalability and formal verification precision to target critical conditions
- Integrate simulation and formal verification.
- Introduce abstract models to overcome limitations of BBOX.

Related Works

Related Works

RISC-V Torture Test:

RISCV-DV:

Fuzzing tools

Related Works

RISC-V Torture Test:

- 👍 Generate random instruction sequences to achieve faster coverage closure
- 👎 Lacks structured verification guidance: limited effectiveness in extreme corner cases

RISCV-DV:

Fuzzing tools

Related Works

RISC-V Torture Test:

- 👍 Generate random instruction sequences to achieve faster coverage closure
- 👎 Lacks structured verification guidance: limited effectiveness in extreme corner cases

RISCV-DV:

- 👍 Support CRV (constraint random verification) and RISC V ISA features
- 👎 Reliance on randomness makes it difficult to cover boundary conditions

Fuzzing tools

Related Works

RISC-V Torture Test:

- 👍 Generate random instruction sequences to achieve faster coverage closure
- 👎 Lacks structured verification guidance: limited effectiveness in extreme corner cases

RISCV-DV:

- 👍 Support CRV (constraint random verification) and RISC V ISA features
- 👎 Reliance on randomness makes it difficult to cover boundary conditions

Fuzzing tools

- 👎 Cannot guarantee coverage of all possible scenarios.

Problems & Issues

Problems & Issues

- False alarms due to environmental discrepancies.
- BBOX issues by excessive complexity or large storage modules.

Formal Guided Test Sequence Optimization Framework

Overall architecture

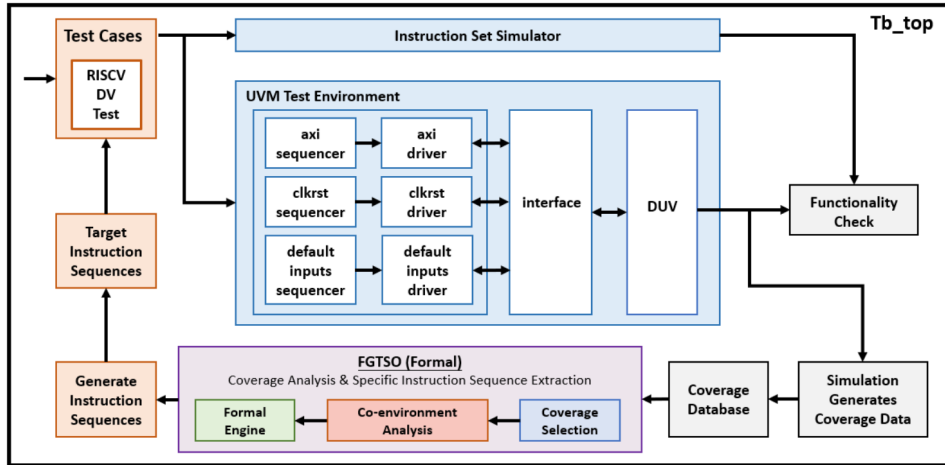


Figure: FGTSO Framework Architecture

Overview

1. Coverage Hole Selection

Overview

1. Coverage Hole Selection
2. Environment Consistency Check

Overview

1. Coverage Hole Selection
2. Environment Consistency Check
3. Formal Cover Property Generation (*assume* properties)

Overview

1. Coverage Hole Selection
2. Environment Consistency Check
3. Formal Cover Property Generation (*assume* properties)
4. Targeted Instruction Sequence Generation

FGTSO Workflow

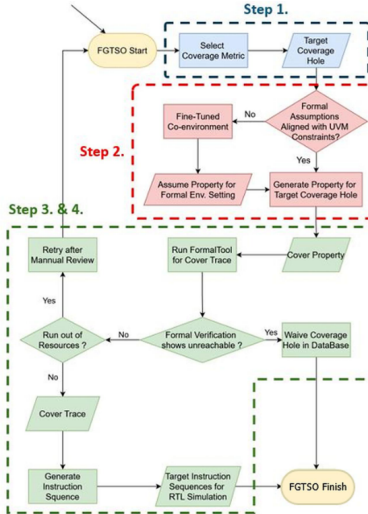


Figure: FGTSO Workflow

FGTSO Workflow - Step 1

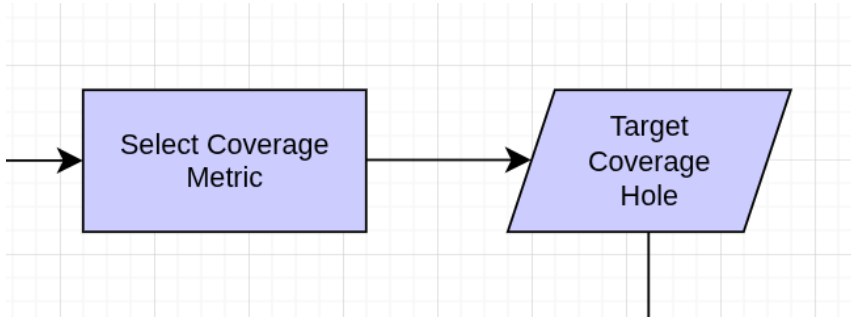


Figure: Step 1: Coverage Hole Selection

FGTSO Workflow - Step 1

Priority: addressing most impactful coverage holes

MaxUncovd strategy

Prioritizes selecting modules with lowest coverage, assuming that addressing the most uncovered areas yields the greatest verification impact.

FGTSO Workflow - Step 2

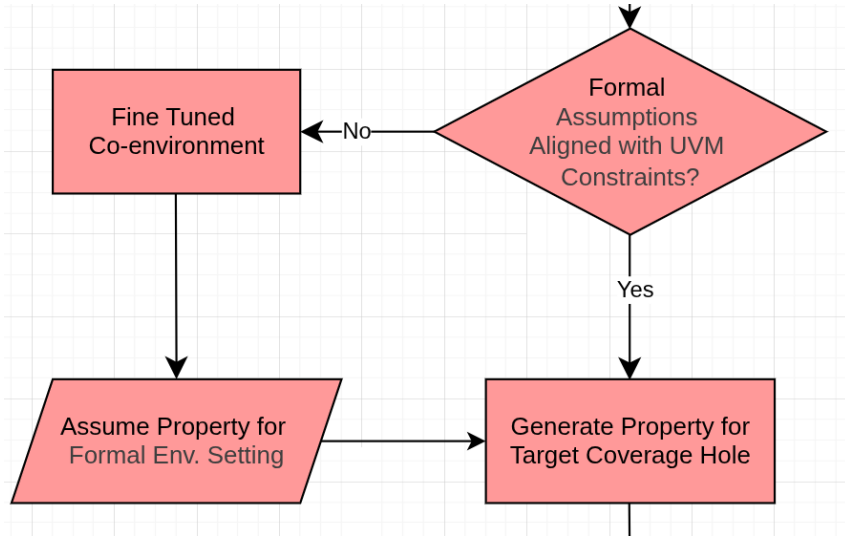


Figure: Step 2: Environment Consistency Check

Coverage Holes Analysis

SCORE	BRANCH	NAME	
86.05	86.05	wt_dcache_ctrl	179 MISS_REQ: begin
			180 miss_req_o = 1'b1;
			181
87.50	87.50	fpnew_opgroup_fmt_slice	182 if (req_port_i.kill_req) begin
			183 req_port_o.data_rvalid = 1'b1;
			184 if (miss_ack_i) begin
88.19	88.19	control_mvp	185 state_d = KILL_MISS;
			186 end else begin
88.24	88.24	controller	187 state_d = KILL_MISS_ACK;
			188 end
90.00	90.00	scoreboard	189 end else if (miss_replay_i) begin
			190 state_d = REPLAY_REQ;
90.00	90.00	bht	191 end else if (miss_ack_i) begin
			192 state_d = MISS_WAIT;
91.67	91.67	fifo_v3	193 end

Figure: Coverage Holes Identification

FGTSO Workflow - Step 2

- Misalignment detection is performed to determine consistency.

FGTSO Workflow - Step 2

- Misalignment detection is performed to determine consistency.
- Formal verification generates instruction sequences targeting specific coverage holes, executed in the simulation.

FGTSO Workflow - Step 2

- Misalignment detection is performed to determine consistency.
- Formal verification generates instruction sequences targeting specific coverage holes, executed in the simulation.
- Results from formal verification and simulation are inconsistent (fail to cover intended coverage holes): misalignment.

FGTSO Workflow Misalignment - PIs

Three are the main causes of misalignment: PIs, BBOX and Insufficient RISC-V-DV constraints.

- Primary Inputs (PIs) (reset, clock, interrupt request, rvalid signal AXI protocol).

FGTSO Workflow Misalignment - Pls

Three are the main causes of misalignment: Pls, BBOX and Insufficient RISC-V-DV constraints.

- Primary Inputs (Pls) (reset, clock, interrupt request, rvalid signal AXI protocol).
 - Assume arbitrary values without constraints.
 - Generated cover traces may fail to address coverage holes.
 - Unconstrained Pls can result in misleading coverage estimation (unreachable states by design)
 - Formal assumptions must be introduced to align formal verification with system behavior.

FGTSO Workflow Misalignment - BBOX Constraints

Problem

Modules with high storage requirements or complex computational logic cannot be fully modeled in formal verification, leading to BBOX constraints.

E.g.: for a cache: huge amount of cache lines. Tracking each line is infeasible.

FGTSO Workflow Misalignment - BBOX Constraints

Problem

Modules with high storage requirements or complex computational logic cannot be fully modeled in formal verification, leading to BBOX constraints.

E.g.: for a cache: huge amount of cache lines. Tracking each line is infeasible.

-
- Solution: Abstraction models preserve critical behavior while ensuring interaction with the system remains verifiable.
- High Storage Example: Cache abstraction using SVA array to track indices. Predefined tag replaces full tag tracking.
- Computation Example: Symbolic computation approach using SVA.

Cache Abstraction Model

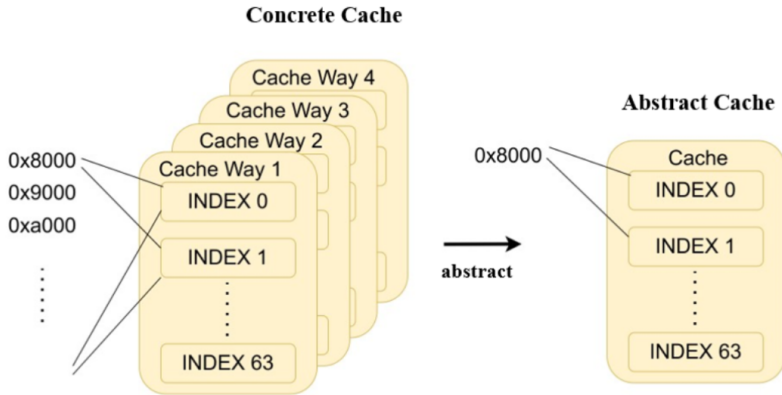


Figure: Cache Abstraction Model

Cache Abstraction Model Codification

```
asm_tag_dcache_idle_0: assume property
(
  if (valid_tag_dcache[gen_cache_wt.i_cache_subsystem.i_wt_dcache.i_wt_dcache_mem.vld_addr])
    ##1 gen_cache_wt.i_cache_subsystem.i_wt_dcache.i_wt_dcache_mem.gen_tag_srams[0].i_tag_sram.rdata_o == 'h100000080000
  else
    ##1 gen_cache_wt.i_cache_subsystem.i_wt_dcache.i_wt_dcache_mem.gen_tag_srams[0].i_tag_sram.rdata_o == 'b0
);
```

Figure: SVA Implementation of Cache Abstraction

FGTSO Workflow Misalignment - RISC-V-DV Constraints

Problem

Formal verification may produce sequences that RISC-V-DV constraints cannot handle.

FGTSO Workflow Misalignment - RISCV-DV Constraints

Problem

Formal verification may produce sequences that RISCV-DV constraints cannot handle.

- Example: Multi-level page table mechanism includes 1 GB, 2 MB, and 4 KB page tables, but original constraints only allow 1 GB page table access.

FGTSO Workflow Misalignment - RISCV-DV Constraints

Problem

Formal verification may produce sequences that RISCV-DV constraints cannot handle.

- Example: Multi-level page table mechanism includes 1 GB, 2 MB, and 4 KB page tables, but original constraints only allow 1 GB page table access.
- Solution: Modify constraints to apply them to addresses pointed to by 1 GB and 2 MB page table entries.

FGTSO Workflow Misalignment - RISCV-DV Constraints

Problem

Formal verification may produce sequences that RISCV-DV constraints cannot handle.

- Example: Multi-level page table mechanism includes 1 GB, 2 MB, and 4 KB page tables, but original constraints only allow 1 GB page table access.
- Solution: Modify constraints to apply them to addresses pointed to by 1 GB and 2 MB page table entries.
- Implementation: Set X, W, and R bits to 0 to indicate that entries should point to the next level of the page table.

FGTSO Workflow Misalignment - RISCV-DV Constraints

Problem

Formal verification may produce sequences that RISCV-DV constraints cannot handle.

- Example: Multi-level page table mechanism includes 1 GB, 2 MB, and 4 KB page tables, but original constraints only allow 1 GB page table access.
- Solution: Modify constraints to apply them to addresses pointed to by 1 GB and 2 MB page table entries.
- Implementation: Set X, W, and R bits to 0 to indicate that entries should point to the next level of the page table.
- Result: Coverage holes for multi-level page table accesses can now be filled.

Step 3 & 4: Formal Cover Property Generation

Step 3 & 4: Formal Cover Property Generation

- Overview: Use formal cover properties to obtain cover traces and generate instruction sequences addressing target coverage holes.
- Property Definition: Step automatically extracts coverage hole expression and applies it to predefined cover templates.
- Formal Execution: Formal tool executes the cover property and derives the input patterns required to satisfy the condition.
- Cover Trace Generation: Generates corresponding cover trace from the formal verification results.
- Sequence Extraction: Extract the instruction sequence from the trace to effectively cover the coverage hole.

Formal Cover Property Example

```
test1: cover property
(
  gen_cache_wt.i_cache_subsystem.i_wt_dcache.gen_rd_ports[1].genblk1.i_wt_dcache_ctrl.state_q == 'h2 &&
  ~gen_cache_wt.i_cache_subsystem.i_wt_dcache.gen_rd_ports[1].genblk1.i_wt_dcache_ctrl.req_port_i.kill_req &&
  gen_cache_wt.i_cache_subsystem.i_wt_dcache.gen_rd_ports[1].genblk1.i_wt_dcache_ctrl.miss_replay_i
);
```

Figure: Cover Property Definition

Handling Coverage Challenges

Handling Coverage Challenges

- Unreachable Coverage Holes: constraints are overly restrictive?
- Waiving Unreachable Holes: If appropriate & hole is unreachable, it is waived; information still logged for debugging.
- Resource Exhaustion: State space explosion.
- Decomposition Strategy: Complex properties decomposed in smaller sub-properties
- Integration: Integrate sub-property traces into comprehensive instruction sequence

Final Outcome

```

main:
    addiw x10, x10, -27
    ld x24, 0(x2)
    slli x8, x8, 8
    addi x8, x2, 0
    csrrw x16, 0x7c1, x2
    lh x15, 4(x2)
    lw x9, 0(x8)
    lhu x3, 1192(x24)
    csrrsi x8, 0x7c1, 3
    lb x28, 1536(x9)
    beq x1, x2, next13632

next13632:
    slli x8, x8, 8
    lbu x0, -2047(x15)
  
```

```

179     MISS_REQ: begin
180         miss_req_o = 1'b1;
181
182         if (req_port_i.kill_req) begin
183             req_port_o.data_rvalid = 1'b1;
184             if (miss_ack_i) begin
185                 state_d = KILL_MISS;
186             end else begin
187                 state_d = KILL_MISS_ACK;
188             end
189         end else if (miss_replay_i) begin
190             state_d = REPLAY_REQ;
191         end else if (miss_ack_i) begin
192             state_d = MISS_WAIT;
193         end
  
```

Figure: FGTSO Results: Coverage Achievement

Comparison with Other Frameworks

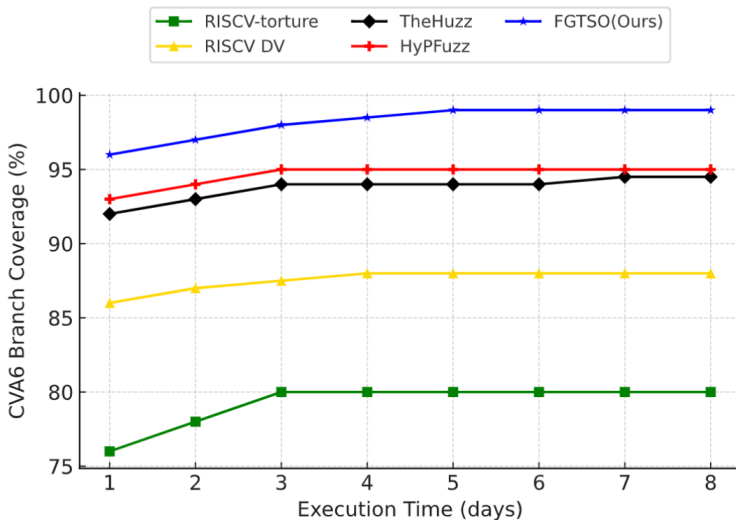


Figure: FGTSO vs. Other Verification Approaches

100% Coverage

Name	Score	Line	Toggle	Condition	Branch
uvmt_cva6_tb	100.00%	100.00%	100.00%	100.00%	100.00%
cva6_dut_wrap	100.00%	100.00%	100.00%	100.00%	100.00%
cva6_tb_wrapper_i	100.00%	100.00%	100.00%	100.00%	100.00%
i_cva6	100.00%	100.00%	100.00%	100.00%	100.00%
commit_sta...	100.00%	100.00%	100.00%	100.00%	100.00%
controller_i	100.00%	100.00%	100.00%	100.00%	100.00%
csr_regfile_i	100.00%	100.00%	100.00%	100.00%	100.00%
ex_stage_i	100.00%	100.00%	100.00%	100.00%	100.00%
gen_cache_...	100.00%	100.00%	100.00%	100.00%	100.00%
i_frontend	100.00%	100.00%	100.00%	100.00%	100.00%
id_stage_i	100.00%	100.00%	100.00%	100.00%	100.00%
issue_stage_i	100.00%	100.00%	100.00%	100.00%	100.00%

Figure: FGTSO: Achieving Complete Coverage

Thanks for your attention!

Questions?

Based on the paper

SHEN, Yu-Shien, et al. Efficient Coverage Optimization with Formal-Guided Testcase Generation in UVM Verification. In: DVCon Europe 2025; Design and Verification Conference and Exhibition. VDE, 2025. p. 43-48.